

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	iv
Índice de figuras	v
Introducción	1
1 Propuestas analizadas	4
1.1 Theseus	5
1.1.1 Nombre del proyecto o sistema operativo	5
1.1.2 Enlace al repositorio y/o documentación oficial	5
1.1.3 Objetivo del proyecto	5
1.1.4 Lenguaje(s) de implementación	5
1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)	6
1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)	8
1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.) . . .	11
1.1.8 Nivel de complejidad y accesibilidad para estudiantes	12
1.2 HelenOS	13
1.2.1 Nombre del proyecto o sistema operativo	13

1.2.2	Enlace al repositorio y/o documentación oficial	13
1.2.3	Objetivo del proyecto	13
1.2.4	Lenguaje(s) de implementación	13
1.2.5	Arquitectura del sistema (monolítica, microkernel, etc.)	14
1.2.6	Componentes implementados (procesos, memoria, archivos, etc.)	16
1.2.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	20
1.2.8	Nivel de complejidad y accesibilidad para estudiantes	21
2	Comparación técnica	22
3	Análisis crítico	23
	Referencias	24

Índice de tablas

1.1	Comparación entre un kernel tradicional y el nanocore de Theseus . .	7
1.2	Funcionamiento técnico (idea clave) de los principales subsistemas de Theseus OS	9
1.3	Funcionamiento técnico (idea clave) de los principales subsistemas de HelenOS	17

Índice de figuras

1.1	Comparativa de la arquitectura de Theseus frente a arquitecturas tradicionales.	8
1.2	Vista general de la arquitectura/organización del kernel de HelenOS.	15
1.3	Árbol de drivers en HelenOS.	19

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 Theseus

1.1.1 Nombre del proyecto o sistema operativo

El nombre del sistema operativo es “Theseus”; según (Theseus OS Project, 2023), fue nombrado así en honor a la paradoja del barco de Teseo, que plantea la cuestión de, si a un barco le renuevas todas sus piezas, ¿Ese barco renovado sigue siendo el mismo barco? Esta idea de renovación de cada componente es característica de este sistema operativo.

1.1.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/theseus-os/Theseus>
- Enlace al libro oficial: <https://www.theseus-os.com/Theseus/book/index.html>

1.1.3 Objetivo del proyecto

Según (Boos et al., 2020), este sistema operativo es en esencia experimental, aunque puede usarse como objeto de estudio debido a su innovadora arquitectura. También tiene objetivos técnicos, como reestructurar la modularidad, reducir el “state spill” y la recuperación ante fallos (fault recovery).

1.1.4 Lenguaje(s) de implementación

Según (Boos et al., 2020) y confirmando con el repositorio oficial (Boos, 2024), el sistema operativo Theseus fue casi completamente desarrollado en Rust, aunque también

se ha usado C para una futura implementación de la librería “libc” dirigida a Theseus (tlibc), ya que por el momento se tomó prestado de la biblioteca de REDOX (relibc).

1.1.5 Arquitectura del sistema (monolítica, microkernel, etc.)

Theseus, como sistema operativo no tiene una arquitectura clásica como monolítica o microkernel, ni mucho menos multikernel; sino que basa su estructura en **cells** o células por su traducción en español, las cuales haciendo honor a su definición biológica, son módulos pequeños e independientes que sirven como bloque de construcción para el sistema operativo. Adicionalmente, theseus tiene un componente mínimo el cual es llamado “nano-core”, que se encarga de iniciar el sistema, establecer memoria mínima y cargar las demás células; pero a diferencia de un kernel propiamente dicho, este no es un jefe permanente, sino que cada cell es independiente (Boos et al., 2020). A continuación una tabla comparativa entre un kernel clásico y el nano-core de theseus:

Tabla 1.1: Comparación entre un kernel tradicional y el nanocore de Theseus

Aspecto	Kernel tradicional	Nanocore (Theseus)
Definición	Es el núcleo del sistema operativo. Centraliza la gestión de componentes y recursos.	Unidad mínima del núcleo que gestiona sólo lo esencial para iniciar y mantener cells.
Estructura	Monolítica o microkernel: el kernel controla los servicios del sistema.	No existe un “núcleo único”: el sistema está compuesto por módulos cooperativos.
Acoplamiento	Alto. Los módulos dependen del kernel y de otros subsistemas.	Bajo. Cada módulo es reemplazable y se comunica mediante interfaces explícitas.
Objetivo	Proveer servicios centrales de manera estable.	Eliminar el <i>state spill</i> (derrame de estado), permite reemplazar componentes en ejecución.

Fuente: Adaptado en base a (Boos et al., 2020), *Theseus: An Experiment in OS Structure for State Management*.

Para entenderlo mejor, se especifica que se impone una arquitectura/organización plana para estas células, todo se corre en un solo SAS (single address space) y un solo nivel de privilegio (no se diferencia entre modo usuario y modo kernel) (Boos and Zhong, 2017). Además, se incluye una figura comparativa entre arquitecturas clásicas y la de theseus basada en células:

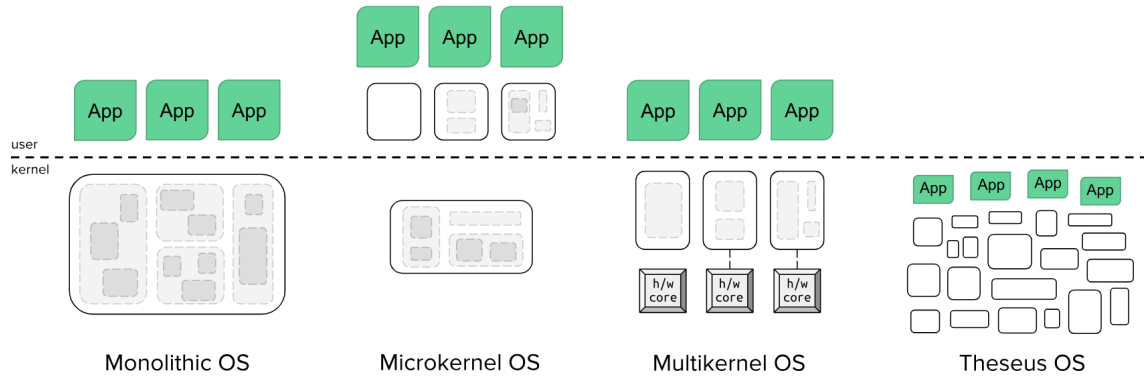


Figura 1.1: Comparativa de la arquitectura de Theseus frente a arquitecturas tradicionales.

Fuente: Obtenido de (Theseus OS Project, 2023) *The Theseus OS Book*.

1.1.6 Componentes implementados (procesos, memoria, archivos, etc.)

En theseus, cada cell encapsula una parte del sistema, encapsula cada componente, como gestión de memoria, planificación/Scheduling, E/S o comunicación. Este trabajo con cells permite aislar y detectar fallos, para poder reemplazar componentes en ejecución y evitar el *state spill* entre módulos (Boos et al., 2020). A continuación y a modo de introducción, se presenta una tabla con los principales componentes y subsistemas de Theseus OS, los nombres de sus archivos en el repositorio oficial y la idea clave detrás de sus funcionamiento.

Tabla 1.2: Funcionamiento técnico (idea clave) de los principales subsistemas de Theseus OS

Componente / Sub-sistema	Nombre en Theseus (archivo/crate)	Funcionamiento técnico (idea clave)
Gestión de memoria	<code>memory crate</code>	Mapea páginas virtuales a marcos físicos garantizando exclusividad.
Carga dinámica de módulos (módulos / células)	<code>mod_mgmt crate</code>	Carga en ejecución módulos (“cells”) y gestiona sus dependencias.
Scheduling / multitarea	<code>scheduler / task crates</code>	Ejecuta tareas en un único espacio de direcciones compartido.
Manejo de archivos / sistema de archivos	<code>fs crate</code>	Gestiona operaciones de archivos, bajo la modularidad de Theseus.
Subsistemas de E/S y drivers	Ej. <code>e1000</code> , <code>ata_pio</code> , <code>keyboard</code>	Controladores modulares escritos en Rust, diseñados para reemplazo.
Recuperación ante fallos / actualización en caliente	<code>loader</code> , <code>spawn crates</code>	Reemplaza módulos activos sin reiniciar el sistema completo.
Comunicación entre módulos	<code>event_types</code> , <code>device_manager crates</code>	Mensajería sin estado entre módulos para evitar dependencias implícitas.

Fuente: Elaboración propia con base en (Boos et al., 2020; Theseus OS Project, 2023; Boos and Zhong, 2017).

Gestión de memoria

El componente `memory` de Theseus implementa una idea `MappedPages`, que representa una región de páginas virtuales adyacentes, mapeadas a marcos físicos. El mapeo se realiza con la función `Mapper::map(pages, frames, flags, pg_tbl)`, y se aprovechan las utilidades de Rust para asegurar que cada página tenga un único marco asociado (Boos et al., 2020; Theseus OS Project, 2023).

Carga dinámica de módulos (módulos / células)

El subsistema `mod_mgmt` permite que los módulos (cells) se carguen, vinculen y descarguen en tiempo de ejecución. Para ello, se construyen instancias de `CrateNamespace`. Este subsistema nos permite sustituir un módulo (cell) sin reiniciar por completo el sistema (Theseus OS Project, 2023).

Scheduling / multitarea

Los crates `scheduler` y `task` gestionan las tareas en un único entorno de espacio de direcciones (SAS) y un solo nivel de privilegio (SPL). Cada tarea se encapsula mediante una estructura `Task` que contiene el contexto del CPU, el puntero de pila y el estado de ejecución. Este enfoque de unicidad evita el fenómeno del “state spill” entre módulos (Boos, 2024).

Manejo de archivos / sistema de archivos

El crate `fs` implementa las funciones básicas del sistema de archivos, como lectura, escritura y gestión de directorios. Siguiendo la filosofía de Theseus, este módulo

se carga como una cell independiente, permitiendo su reemplazo o actualización sin necesidad de reiniciar el sistema (Boos, 2024).

Subsistemas de E/S y drivers

Los drivers como `e1000`, `ata_pio` o `keyboard` escritos en Rust; gestionan hardware de red, discos PATA/IDE y teclados PS/2 respectivamente. Fiel al enfoque de Theseus, son independientes y reemplazables en tiempo de ejecución; para facilitar el “fault recovery” (Boos, 2024).

Recuperación ante fallos / actualización en caliente

Los crates `loader` y `spawn` son las cells que permiten el tan aclamado “fault recovery” y “live update”; o sea recuperación ante fallos y actualización, de cells, sin reiniciar todo el sistema. El funcionamiento contiene detención de tareas, liberación de recursos y recarga de cells (Theseus OS Project, 2023).

Comunicación entre módulos

Los subsistemas `event_types` y `device_manager` se encargan de la correcta mensajería entre cells mediante canales tipados y sin depender de estado compartido. Las dependencias se definen con metadatos para eliminar el “state spill” en la comunicación entre componentes (Boos and Zhong, 2017).

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

Theseus emplea y modifica herramientas de y para Rust; también utiliza máquinas virtuales. Las principales herramientas son las siguientes:

- **Rust toolchain:** Uso de `cargo`, `rustc` y `crates` estándar del ecosistema Rust para la compilación, gestión de dependencias y construcción modular de Theseus.
- **Sistema de carga dinámica y enlazado en tiempo de ejecución:** Theseus modifica el compilador (`rustc`) y el “linker” para permitir la carga y sustitución de (cells) en ejecución (Boos and Zhong, 2017).
- **Emuladores y máquinas virtuales:** Se utilizan entornos como “QEMU” para probar, depurar y validar el funcionamiento del sistema (Boos, 2024).
- **Librería estandar de C:** Actualmente, Theseus utiliza gran parte de `relibc` de Redox OS como su biblioteca estándar de C, pero planea desarrollar su propia versión llamada `tlibc` (Theseus lib) en el futuro (Boos, 2024).

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

Considerando un entorno académico, que es justamente donde rust brilla, es un proyecto digno de estudio, con un altísimo nivel de complejidad debido también y en mayor medida, a su arquitectura basada en cells; también cabe mencionar que la dificultad sube debido al uso de la versión de la librería estandar de C (`relibc`) y el uso de QEMU, el cual es un emulador no tan intuitivo y del cual no hay tanta documentación en español.

Sin embargo, se ve compensado debido a su alta accesibilidad, la cual es posible gracias a su extensa documentación y estudios en torno a este proyecto, propiciados por el mismo creador de Theseus.

1.2 HelenOS

1.2.1 Nombre del proyecto o sistema operativo

El nombre del sistema operativo es “HelenOS”. El proyecto fue iniciado por Jakub Jermar en 2001, en ese momento era un código independiente que funcionaba como kernel para IA-32. (Děcký, 2015)

1.2.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/HelenOS/helenos>
- Documentación oficial y Wiki: <https://www.helenos.org/>

1.2.3 Objetivo del proyecto

Según (Děcký, 2015), este sistema operativo, tiene un enfoque educativo y experimental. Además, busca servir como una plataforma para la investigación y desarrollo de sistemas operativos de propósito general, teniendo muy en cuenta la fiabilidad y practicidad.

1.2.4 Lenguaje(s) de implementación

Según (Děcký, 2010) y confirmando con el repositorio oficial (Jermář, 2025), el sistema operativo HelenOS está principalmente implementado en C, con algo de apoyo de C++, Meson como “build system”, algunos componentes escritos en ensamblador para tareas de bajo nivel y python para scripts auxiliares.

1.2.5 Arquitectura del sistema (monolítica, microkernel, etc.)

El sistema operativo HelenOS se basó en la arquitectura multiserver de microkernel, multiserver es un tipo de arquitectura microkernel, donde el núcleo del sistema operativo (microkernel) es responsable de las funciones básicas para el funcionamiento del sistema operativo como gestión de memoria, comunicación de procesos y scheduling; y por otro lado, los demás servicios del sistema operativo se implementan como “servidores” y se ejecutan en el espacio de usuario. De esta manera, los servidores pueden ser desarrollados, cambiados y reiniciados de manera independiente sin afectar al kernel (Děcký, 2015). A continuación, se incluye una vista general de la arquitectura/organización del kernel de HelenOS:

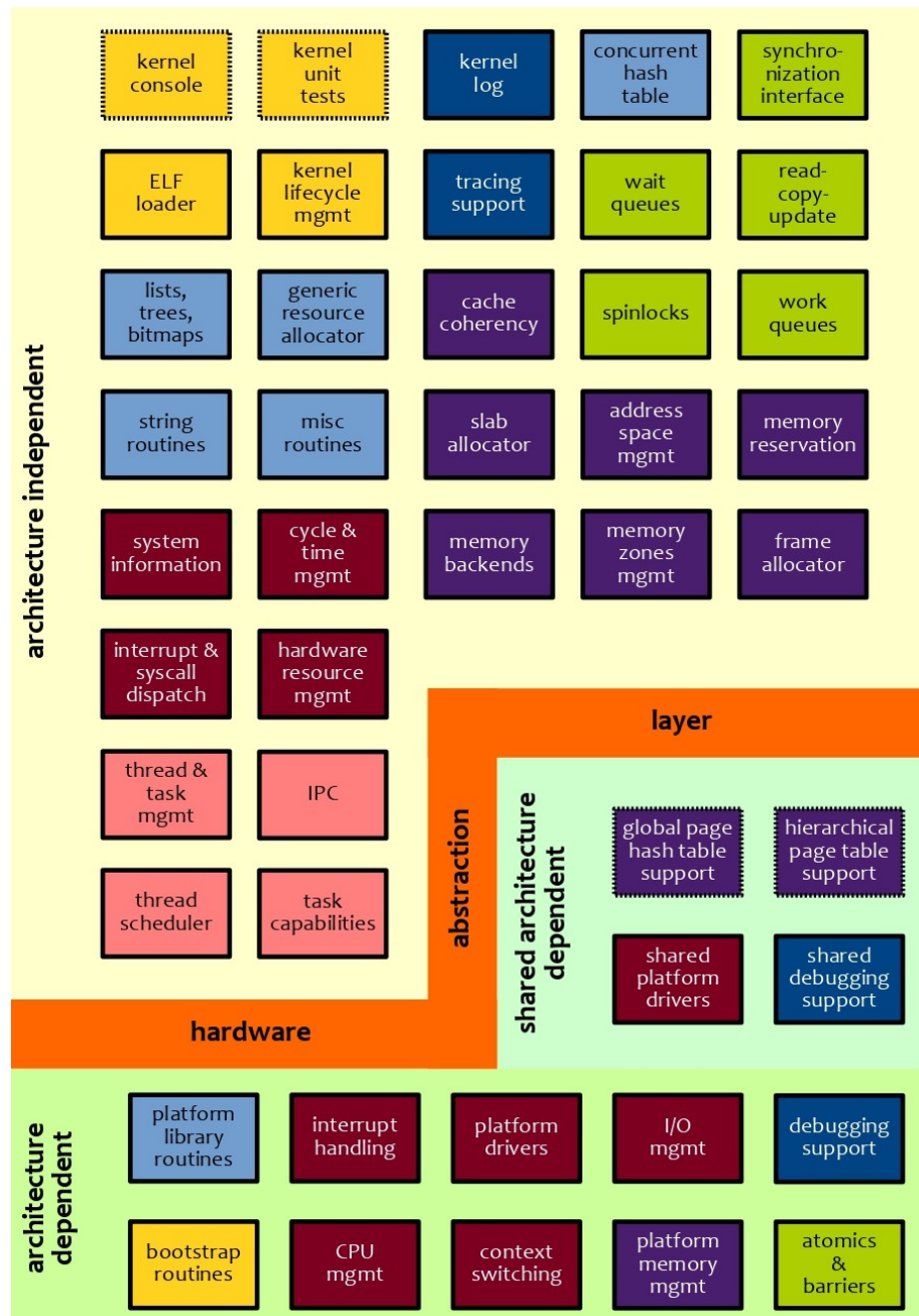


Figura 1.2: Vista general de la arquitectura/organización del kernel de HelenOS.

Fuente: Obtenido de (Děcký, 2015) *Application of Software Components in Operating System Design*.

1.2.6 Componentes implementados (procesos, memoria, archivos, etc.)

En HelenOS, cada componente del sistema operativo se implementa como una tarea aislada con su propio espacio de direcciones (siguiendo la arquitectura multiserver), identificador único y comunicación mediante interfaces bien definidas (IPC o syscall API) (Děcký, 2015). A continuación se presenta una tabla con los principales componentes y subsistemas de HelenOS, su nombre de archivo en el repositorio oficial y la idea técnica detrás de su implementación.

Tabla 1.3: Funcionamiento técnico (idea clave) de los principales subsistemas de HelenOS

Componente / Sub-sistema	Nombre en HelenOS (tarea / servicio)	Funcionamiento técnico (idea clave)
Gestión de memoria	frame allocator, slab allocator, address space mgmt	Asignación de marcos físicos con separación por tareas.
Planificación / multitarea	thread scheduler, task mgmt	Aislamiento por espacio de direcciones y comunicación por IPC.
Sistema de archivos	TMPFS, FAT, ext4, MINIX FS	Implementados como tareas independientes, comunicadas vía VFS.
Subsistemas de E/S y drivers	Ej. rtl8139, ahci, ps2, usb hid	Se usan como tareas separadas, organizados en el árbol de dispositivos.
Servicios de red	inetsrv, tcp, udp, ethip	Componentes aislados, comunicados por IPC.
Servicios de usuario / consola	console, clipboard, compositor, bdsh	Tareas independientes con comunicación por IPC.
Servicios de nombres y localización	naming service, location service	Servicios singleton para identificar y vincular componentes.

Fuente: Elaboración propia con base en (Děcký, 2015; Jermář, 2025).

Gestión de memoria

En HelenOS, se implementan los componentes `frame allocator`, `slab allocator` y `address space management` para asignar marcos físicos, gestionar zonas de memoria y mantener espacios de direcciones separados por tarea. Esta separación es para encapsular el estado de cada componente y facilitar la verificación de corrección (Děcký, 2015, p. 39).

Planificación / multitarea

La planificación se realiza mediante `thread scheduler` y `task management`. Le generan a cada tarea su propio espacio de direcciones y permiten su comunicación con otras mediante IPC (Děcký, 2015, p. 38).

Sistema de archivos

HelenOS soporta múltiples sistemas de archivos como `TMPFS`, `FAT`, `ext4`, `MINIX FS`, cada uno implementado como una tarea independiente; se comunican con el sistema mediante el `vfs`, el cual es un intermediario entre drivers y aplicaciones (Děcký, 2015, p. 40).

Subsistemas de E/S y drivers

Los drivers están organizados jerárquicamente en el árbol de dispositivos (como se muestra en la figura). Cada driver (como `rtl8139`, `ahci`, `ps2`, `usb hid`) se implementa como una tarea separada, para permitir modularidad, aislamiento y reemplazo en ejecución (Děcký, 2015, p. 41).

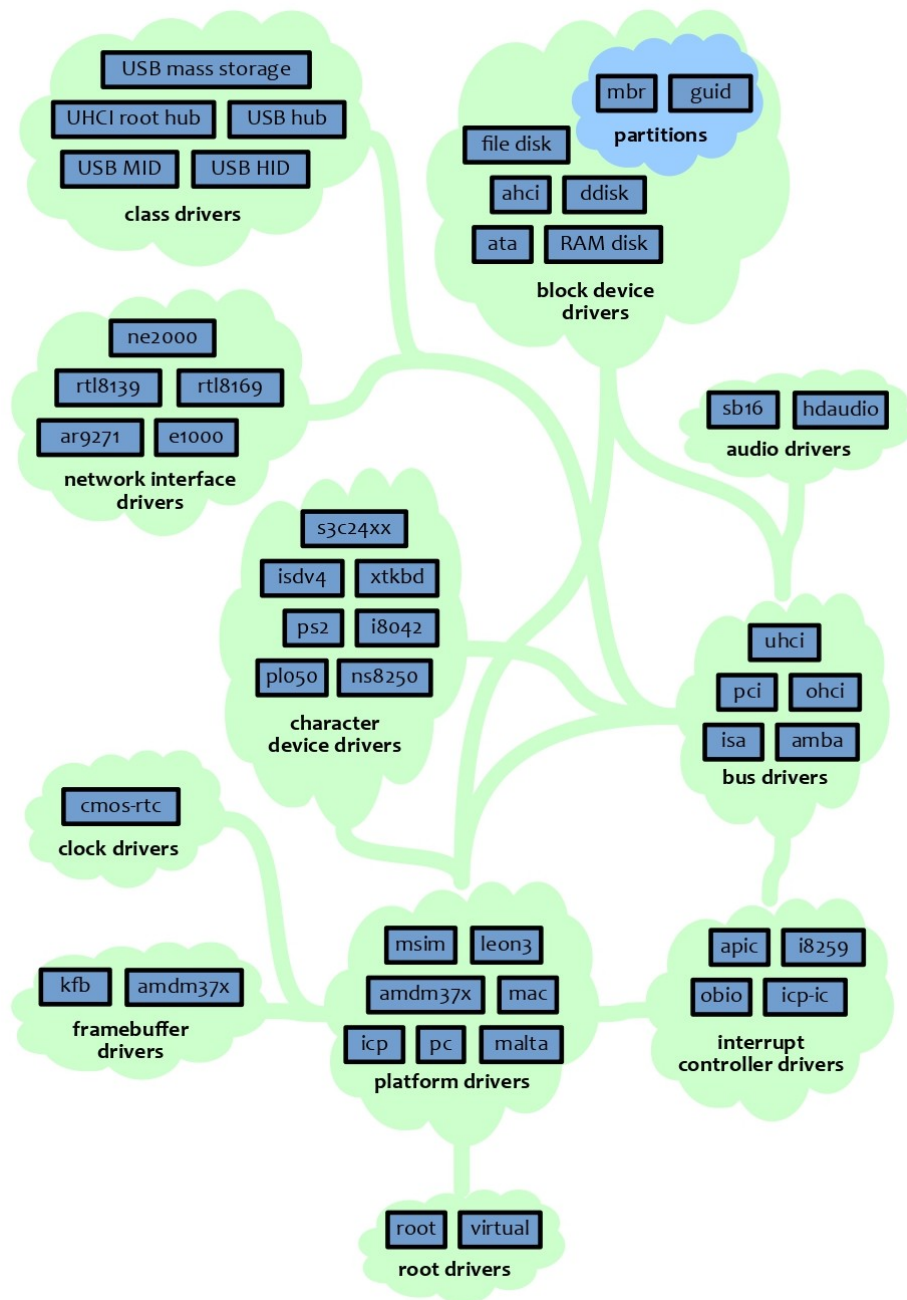


Figura 1.3: Árbol de drivers en HelenOS.

Fuente: Obtenido de (Děcký, 2015) *Application of Software Components in Operating System Design*.

Servicios de red

Los protocolos de red `tcp`, `udp`, `ethip` y el servicio `inetsrv` se implementan como componentes independientes. Siguiendo así la arquitectura multiserver (Děcký, 2015, p. 40).

Servicios de usuario / consola

Los componentes `console`, `clipboard`, `compositor` y el shell `bdsh` se ejecutan como tareas separadas (Děcký, 2015, p. 40).

Servicios de nombres y localización

Son servicios singleton, específicamente `naming service` y el `location service`, que permiten identificar y vincular componentes en el sistema (Děcký, 2015, p. 38).

1.2.7 Herramientas utilizadas (compiladores, emuladores, etc.)

HelenOS utiliza herramientas para el lenguaje C, también usa máquinas virtuales, sistemas de construcción e implementaciones. Entre sus principales herramientas están:

- **Compilador:** Uso de GCC y Clang, con toolchains cruzadas para las diferentes arquitecturas soportadas (x86, ARM, MIPS, SPARC, PowerPC e Itanium) (Jermář, 2025).
- **Emuladores y máquinas virtuales:** Para ejecutar HelenOS se usa el entorno “QEMU”, para probar, depurar y validar el ISO del microkernel (Jermář, 2025).

- **Sistema de construcción:** Makefile con scripts Bash, Python y uso de MESON (experimental) para apoyar la compilación cruzada de componentes (Jermář, 2025).
- **Herramienta de portabilidad:** *HelenOS Coastline*, un repositorio desarrollado por los creadores del microkernel, puede ser usado para la integración de software externo más fácilmente.

1.2.8 Nivel de complejidad y accesibilidad para estudiantes

Considerando un entorno de estudio de sistemas operativos, justamente el entorno en el que se propicia esta investigación, con conocimientos previos de C y fundamentos de sistemas operativos; es un proyecto con un nivel de complejidad media, más que nada por su tamaño (es extenso), además del uso de servicios mediante IPC y también considerar que la arquitectura multiserver, que implica microkernel, no es la más sencilla tampoco. Y eso sin contar la diversidad de arquitecturas soportadas, lo cual añade dificultad al análisis del código.

Se consideró en la categorización del nivel de complejidad que, la accesibilidad para los estudiantes es bastante buena, ya que el proyecto es open source y cuenta con una gran cantidad de estudios en torno a él. Sin embargo, una consideración a tomar en cuenta es que, aunque HelenOS dispone de documentación formal (guías, artículos, documentación generada), esta puede no detallar cada componente del sistema operativo con la exhaustividad que otros sistemas operativos grandes ofrecen (por ejemplo un libro completo o wiki ultra-detallada).

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

- Boos, K. (2024). Theseus — source code (github repository). <https://github.com/theseus-os/Theseus>. Repositorio accedido el 22 Octubre 2025.
- Boos, K., Liyanage, N., Ijaz, R., and Zhong, L. (2020). Theseus: an experiment in operating system structure and state management. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1–19. USENIX Association.
- Boos, K. and Zhong, L. (2017). Theseus: a state spill-free operating system. In *Proceedings of the ACM Workshop on Programming Languages and Operating Systems (PLOS '17)*. ACM.
- Děcký, M. (2010). A road to a formally verified general-purpose operating system. In *Proceedings of the International Symposium on Architecting Critical Systems (ISARCS '10), Lecture Notes in Computer Science, vol. 6150*. Recuperado el 22 Octubre 2025.
- Děcký, M. (2015). *Application of Software Components in Operating System Design*. PhD thesis, Charles University in Prague, Faculty of Mathematics and Physics.

Jermář, J. (2025). Helenos — source code (github repository). <https://github.com/HelenOS/helenos>. Repositorio accedido el 22 Octubre 2025.

Theseus OS Project (2023). The theseus os book. Recuperado de <https://www.theseus-os.com/Theseus/book/index.html>.